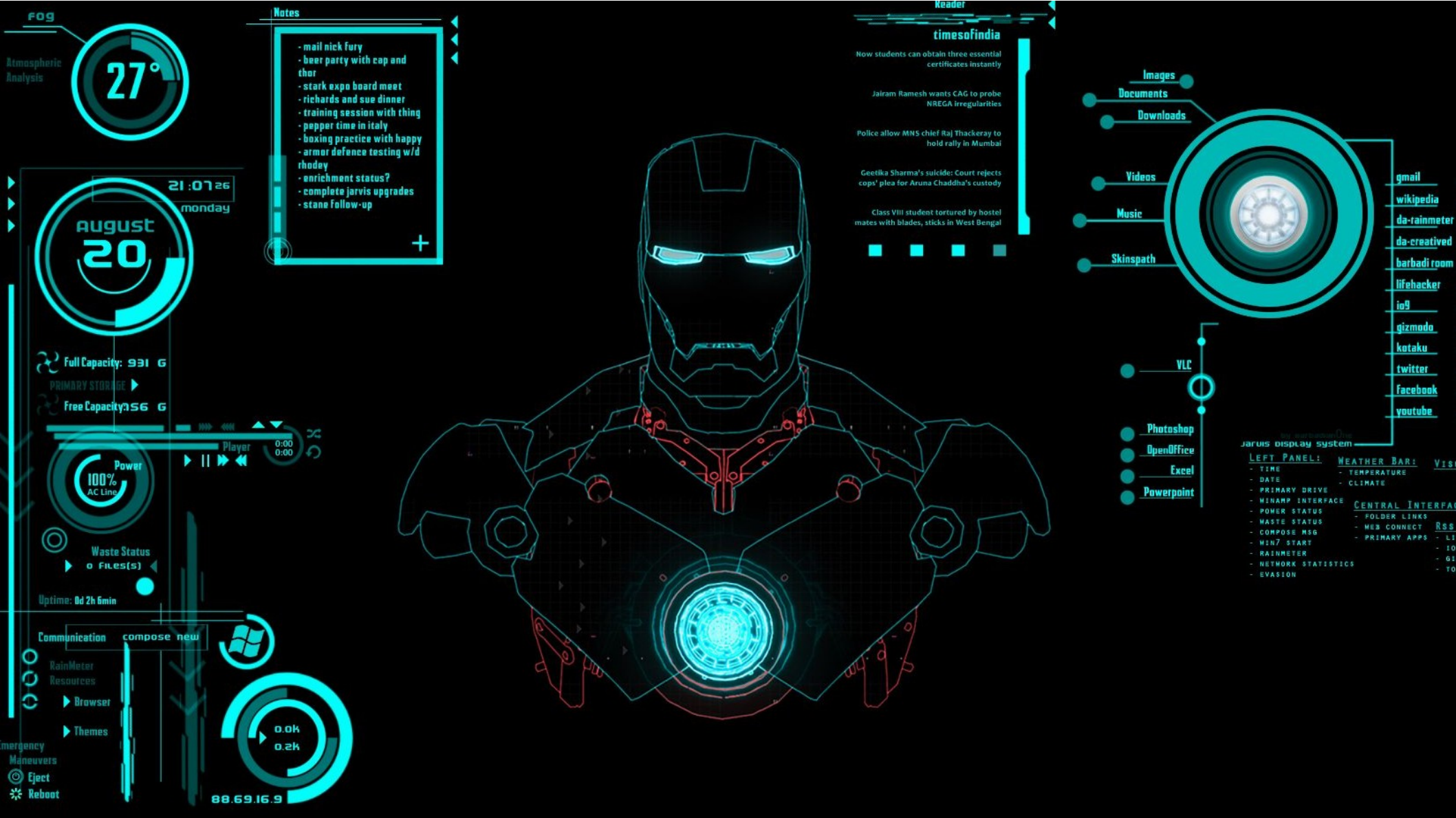




15.Cores e Iluminação Parte 1

Prof. Alexandre Krohn

Roteiro

- Cores
- Iluminação Básica
 - Iluminação Ambiente
 - Iluminação Difusa

Cores

- Utilizamos cores até aqui sem definí-las propriamente
- Cores e iluminação estão diretamente relacionadas

Cores

- No **mundo real**, as cores podem assumir **qualquer valor de cor conhecido**, com cada objeto tendo sua(s) própria(s) cor(es)
- No **mundo digital**, precisamos mapear as (infinitas) cores reais para **valores digitais (limitados)** e, portanto, nem todas as cores do mundo real podem ser representadas digitalmente.

Cores

- As cores são representadas digitalmente usando um componente vermelho, verde e azul comumente abreviado como **RGB**.
- Usando diferentes combinações apenas desses 3 valores, dentro de um intervalo de **[0,1]**, podemos representar quase qualquer cor que existe.

Cores - RGB

- A variação de 0 até 1, ou do nada até 100% pode ser representada digitalmente de diversas formas
- Uma das mais comuns é representar cada componente como um número inteiro de 8 bits:

Mais escuro

$(0, 0, 0) \rightarrow (127, 127, 127) \rightarrow (255, 255, 255)$

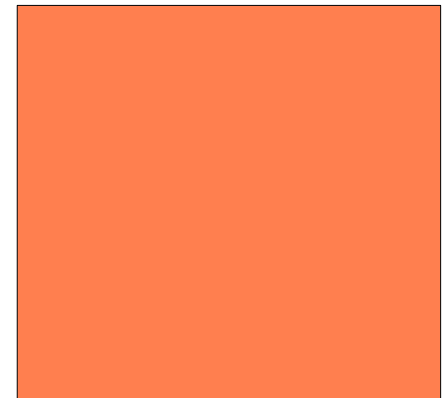
Mais claro

Cores - RGB

- Em OpenGL, usa-se a variação de cada componente de cor como um float entre 0 e 1:
- Por exemplo, para obter uma cor de **coral**, definimos um vetor de cor como:

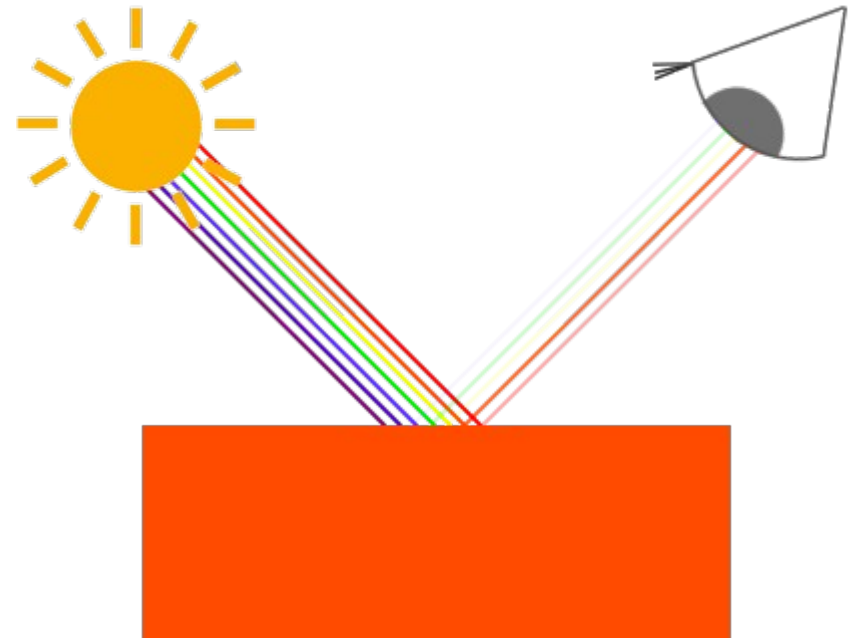
```
glm::vec3 coral(1.0f, 0.5f, 0.31f);
```

Computação Gráfica



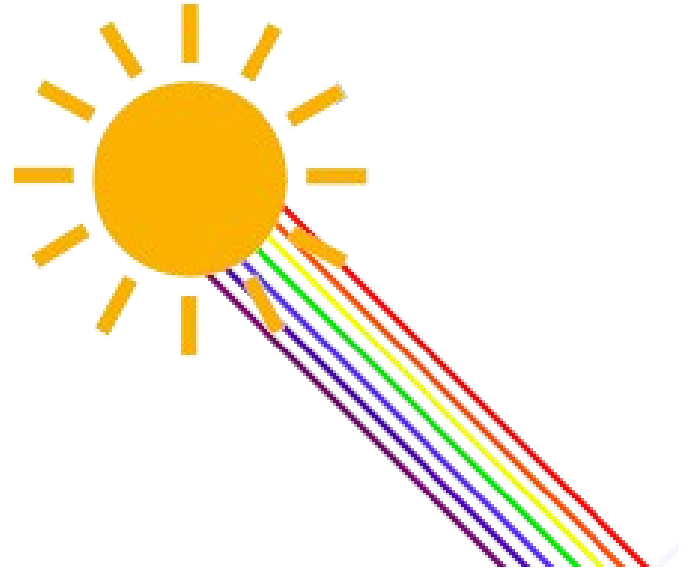
Cores percebidas

- A **cor de um objeto** que vemos na vida real não é a cor que ele realmente tem, mas é a **cor refletida** do objeto.
- As cores que não são absorvidas (rejeitadas) pelo objeto são as cores que percebemos dele.



Cores percebidas

- Por exemplo, a **luz do sol** é percebida como uma luz branca que é a **soma combinada de muitas cores diferentes** (como você pode ver na imagem).



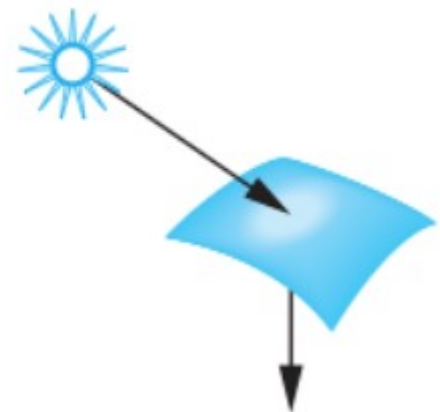
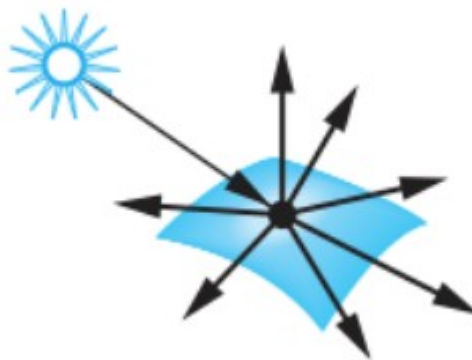
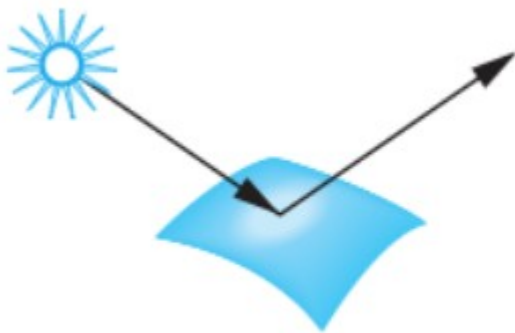
- Se brilharmos essa luz branca em um objeto **azul**, ela absorverá todas as subcores da cor branca, exceto a cor azul.

Computação Gráfica



Cores percebidas

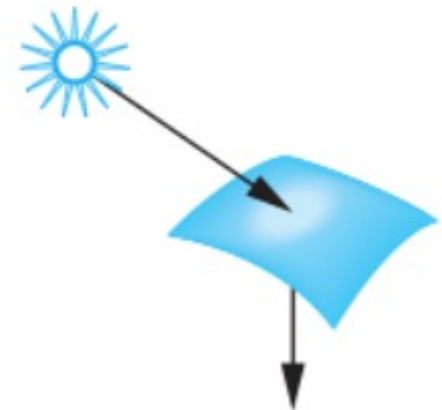
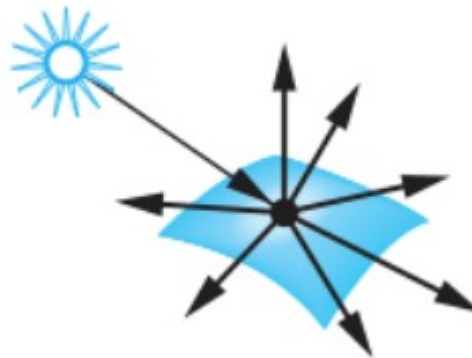
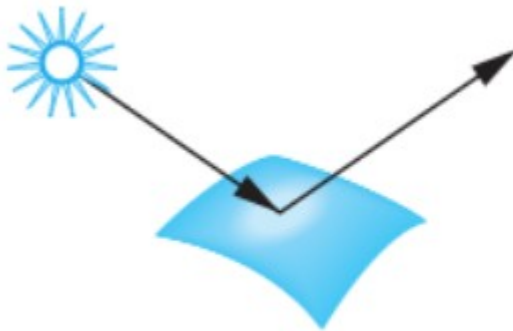
- A luz que atinge a superfície de um objeto pode ser, em maior ou menor intensidade:
 - Refletida
 - Transmitida
 - Absorvida.



Cores percebidas

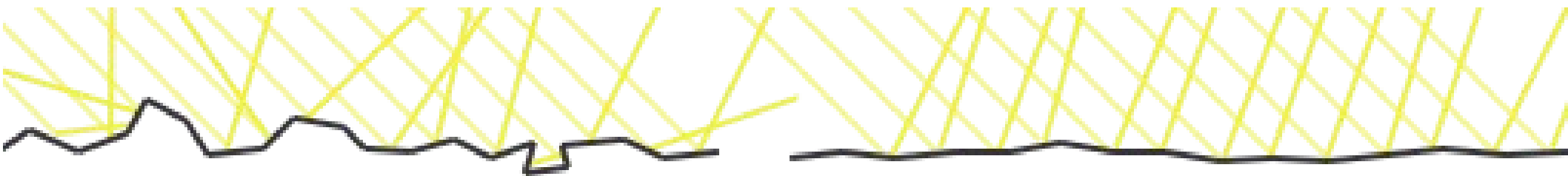
- A luz que atinge a superfície de um objeto pode ser, em maior ou menor intensidade:
 - Refletida
 - Transmitida
 - Absorvida.

Isso depende das
características dos materiais



Cores e materiais

- **Objetos rugosos** tendem a refletir a luz em todas as direções. A luz espalhada é denominada **reflexão difusa** e aparenta ter o mesmo brilho de qualquer ponto de vista. A **cor do objeto** neste caso é a **cor da reflexão difusa** da luz incidente.
- **Objetos lisos** ou polidos refletem mais luz. Essa reflexão é chamada **reflexão especular**, e **depende da posição do observador**



Superfície Rugosa

omputação Gráfic

Superfície Lisa

Cores no OpenGL

- Quando definimos uma **fonte de luz** em **OpenGL**, precisamos definir uma **cor** para a mesma. Se não fizermos isso, o padrão é luz branca
- Os **objetos** também possuem sua própria **cor**
- Como a cor percebida é a **cor refletida** pelo objeto, precisamos calcular a cor percebida

Cores no OpenGL

- **Desconsiderando-se** o tipo de superfície, a cor refletida pode ser calculada da seguinte forma:

```
glm::vec3 lightColor(1.0f, 1.0f, 1.0f); // Branco
glm::vec3 objectColor(1.0f, 0.5f, 0.31f); // Coral
glm::vec3 result = lightColor * objectColor; // = (1.0f, 0.5f, 0.31f);
```

- Multiplicando o vetor de cor da luz incidente pelo vetor de cor do objeto

Cores no OpenGL

- **Desconsiderando-se** o tipo de superfície, a cor refletida pode ser calculada da seguinte forma:

```
glm::vec3 lightColor(1.0f, 1.0f, 1.0f); // Branco
glm::vec3 objectColor(1.0f, 0.5f, 0.31f); // Coral
glm::vec3 result = lightColor * objectColor; // = (1.0f, 0.5f, 0.31f);
```



Note que como a fonte de luz é branca, o objeto refletirá sua própria cor, coral

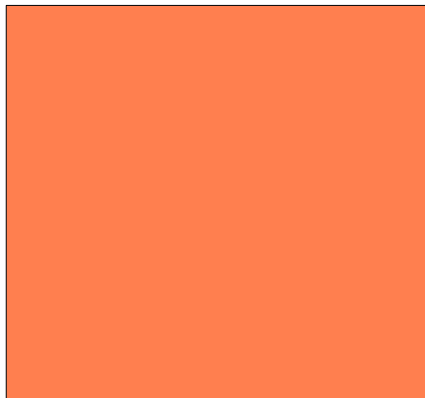
Cores no OpenGL

- Se ao invés de luz branca usássemos uma luz verde?

```
glm::vec3 lightColor(0.0f, 1.0f, 0.0f); // Verde  
glm::vec3 objectColor(1.0f, 0.5f, 0.31f); // Coral  
glm::vec3 result = lightColor * objectColor; // = (0.0f, 0.5f, 0.0f);
```



X



=



Cores no OpenGL

- Como podemos ver, o objeto não possui luz vermelha e azul para absorver e / ou refletir. (Multiplicação por zero)
- O objeto também absorve metade do valor verde da luz, e reflete metade do valor verde da luz.
- A cor do objeto que percebemos seria então uma cor esverdeada escura.
- Podemos ver que se usarmos uma luz verde, apenas os componentes da cor verde podem ser refletidos e, portanto, percebidos; nenhuma cor vermelha e azul é percebida.
- Como resultado, o objeto coral se torna um objeto esverdeado escuro

Computação



The diagram illustrates a color multiplication operation. It consists of three squares arranged horizontally, separated by a multiplication sign (X) and an equals sign (=). The first square is bright green, the second is orange, and the third is dark green. The word 'Computação' is written to the left of the first square.

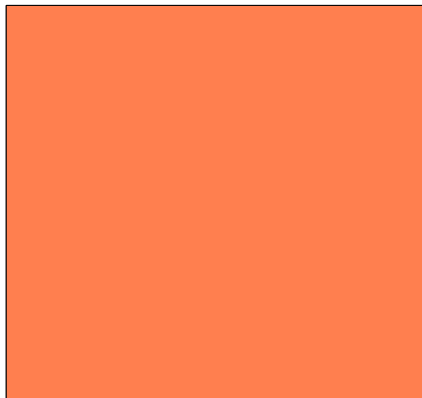
Cores no OpenGL

- E se a luz fosse verde oliva?

```
glm::vec3 lightColor(0.33f, 0.42f, 0.18f); // Verde-oliva  
glm::vec3 objectColor(1.0f, 0.5f, 0.31f); // Coral  
glm::vec3 result = lightColor * objectColor; // = (0.33f, 0.21f, 0.06f);
```



X

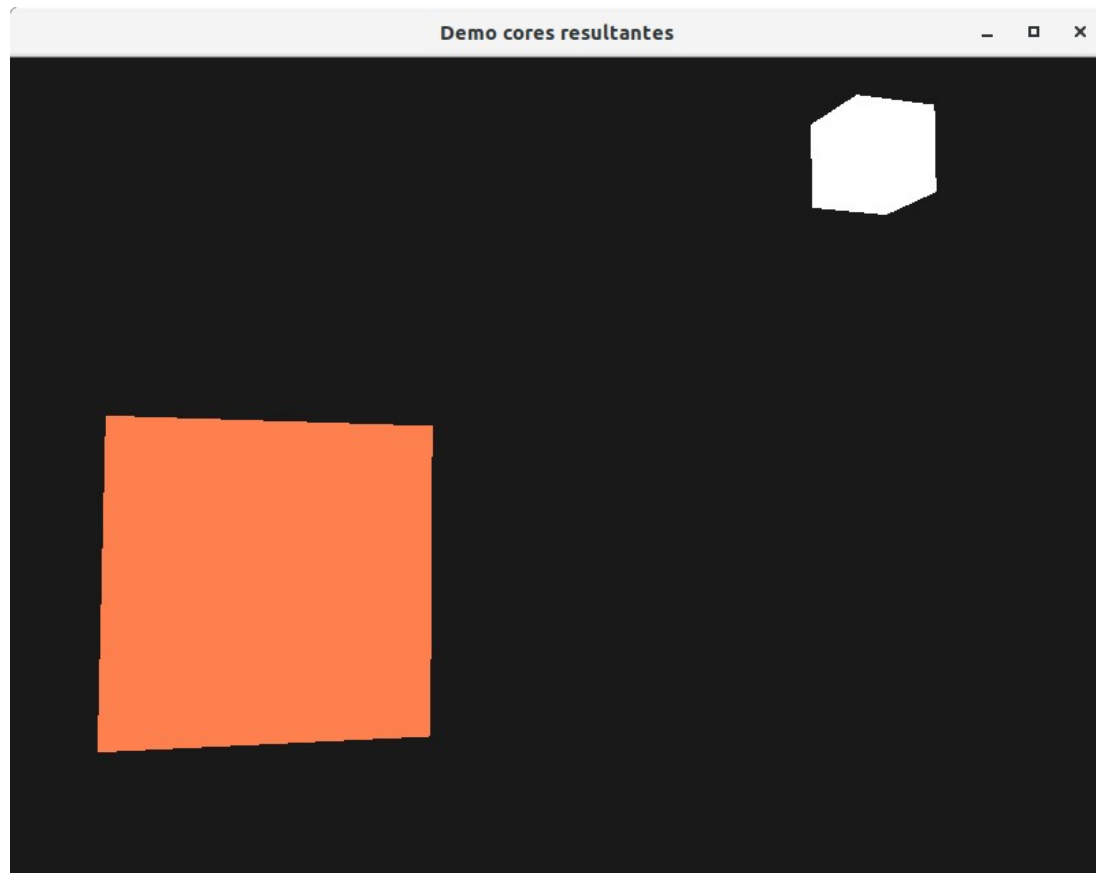


=



Exemplo

- O projeto **GLFW_Iluminacao_0** demonstra o conceito de cor resultante da incidência da luz



- As teclas **R** e **F** alteram a quantidade de luz **vermelha**
- As teclas **T** e **G** alteram a quantidade de luz **verde**
- As teclas **Y** e **H** alteram a quantidade de luz **azul**
- A tecla **ESPAÇO** volta à cor original
- Use o **mouse** e as teclas **W**, **A**, **S** e **D** para movimentar-se pela cena

O código-fonte

- O código-fonte do exemplo possui alguns aspectos que precisam nossa **atenção**:
- A posição do cubo de luz serve **apenas para ilustrar** de qual direção a luz está vindo e qual sua cor.
- Muitas vezes iluminaremos cenas **sem mostrar** explicitamente a fonte de luz.

Vértices e Buffers

- Os vértices foram definidos uma única vez para desenhar um cubo.
- No entanto, foram utilizados para desenhar dois cubos, um representando o objeto e outro a fonte de luz
- Isso é possível porque foram criados dois VAOs e um VBO, e o mesmo VBO foi vinculado aos dois VAOs

Vértices e Buffers

- As posições e tamanhos dos dois cubos são diferentes porque o cubo representando a fonte de luz sofreu transformações de escala e translação, como pode ser observado no trecho a seguir (linha 205)

...

```
model = glm::mat4(1.0f);  
model = glm::translate(model, lightPos);  
model = glm::scale(model, glm::vec3(0.2f)); // a smaller cube  
lightCubeShader.setMat4("model", model);
```

....

Shaders e Câmera

- Os shaders e a lógica da câmera virtual foram abstraídos para **classes C++**, nos arquivos **shader_m.h** e **camera.h**
- Isso distribui a complexidade do código, simplificando o main.
- Isso também permite que shaders diferentes sejam utilizados para objetos diferentes.
- **Essa técnica pode ser usada para aplicar texturas diferentes à diferentes objetos**

Roteiro

- Cores
- Iluminação Básica
 - Iluminação Ambiente
 - Iluminação Difusa

Iluminação Básica

- A iluminação no mundo real é extremamente complicada e depende de muitos fatores, algo que não podemos calcular com o poder de processamento limitado que temos.
- A iluminação em **OpenGL** é, portanto, **baseada em aproximações da realidade** usando modelos simplificados, baseados na física, que são muito mais fáceis de processar e parecem relativamente semelhantes.

Iluminação Básica

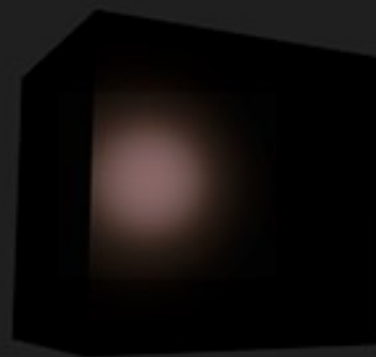
- Um desses modelos é chamado de modelo de iluminação **Phong**.
- Os principais blocos de construção do modelo de iluminação **Phong** consistem em 3 componentes:
 - Iluminação ambiente;
 - Iluminação difusa e
 - Iluminação especular.



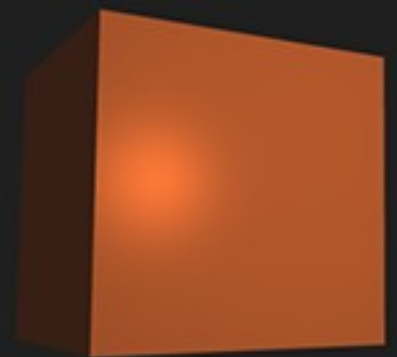
ambient



diffuse



specular



combined (Phong)

Modelo de Phong

- O modelo de **Phong** é um dos modelos de iluminação mais utilizados na computação gráfica devido a sua simplificação das interações entre luz e material e o bom resultado produzido por esta técnica.

Modelo de Phong

- **Iluminação ambiente:** mesmo quando está escuro, geralmente ainda há alguma luz em algum lugar do mundo (a lua, uma luz distante), então os objetos quase nunca ficam completamente escuros. Para simular isso, usamos uma constante de iluminação ambiente que sempre dá ao objeto alguma cor.
- **Iluminação difusa:** simula o impacto direcional que um objeto de luz tem sobre um objeto. Este é o componente mais significativo visualmente do modelo de iluminação. Quanto mais uma parte de um objeto está voltada para a fonte de luz, mais brilhante ela se torna.
- **Iluminação especular:** simula o ponto brilhante de uma luz que aparece em objetos brilhantes. Os realces especulares são mais inclinados à cor da luz do que à cor do objeto.

Roteiro

- Cores
- Iluminação Básica
 - Iluminação Ambiente
 - Iluminação Difusa

Iluminação Ambiente

- A luz geralmente não vem de uma única fonte, mas de muitas fontes espalhadas ao nosso redor, mesmo quando não são imediatamente visíveis.
- Uma das propriedades da luz é que ela pode se espalhar e refletir em várias direções, atingindo pontos que não são diretamente visíveis; a luz pode, portanto, refletir em outras superfícies e ter um impacto indireto na iluminação de um objeto.
- Algoritmos que levam isso em consideração são chamados de algoritmos de iluminação global, mas são complicados e caros de calcular.

Iluminação Ambiente

- Adicionar iluminação ambiente à cena é razoavelmente fácil.
- Pega-se a cor da luz, multiplica-se por um pequeno fator de ambiente constante, multiplica-se o resultado obtido pela cor do objeto e usa-se o valor calculado como a cor do shader de fragmento do objeto cubo

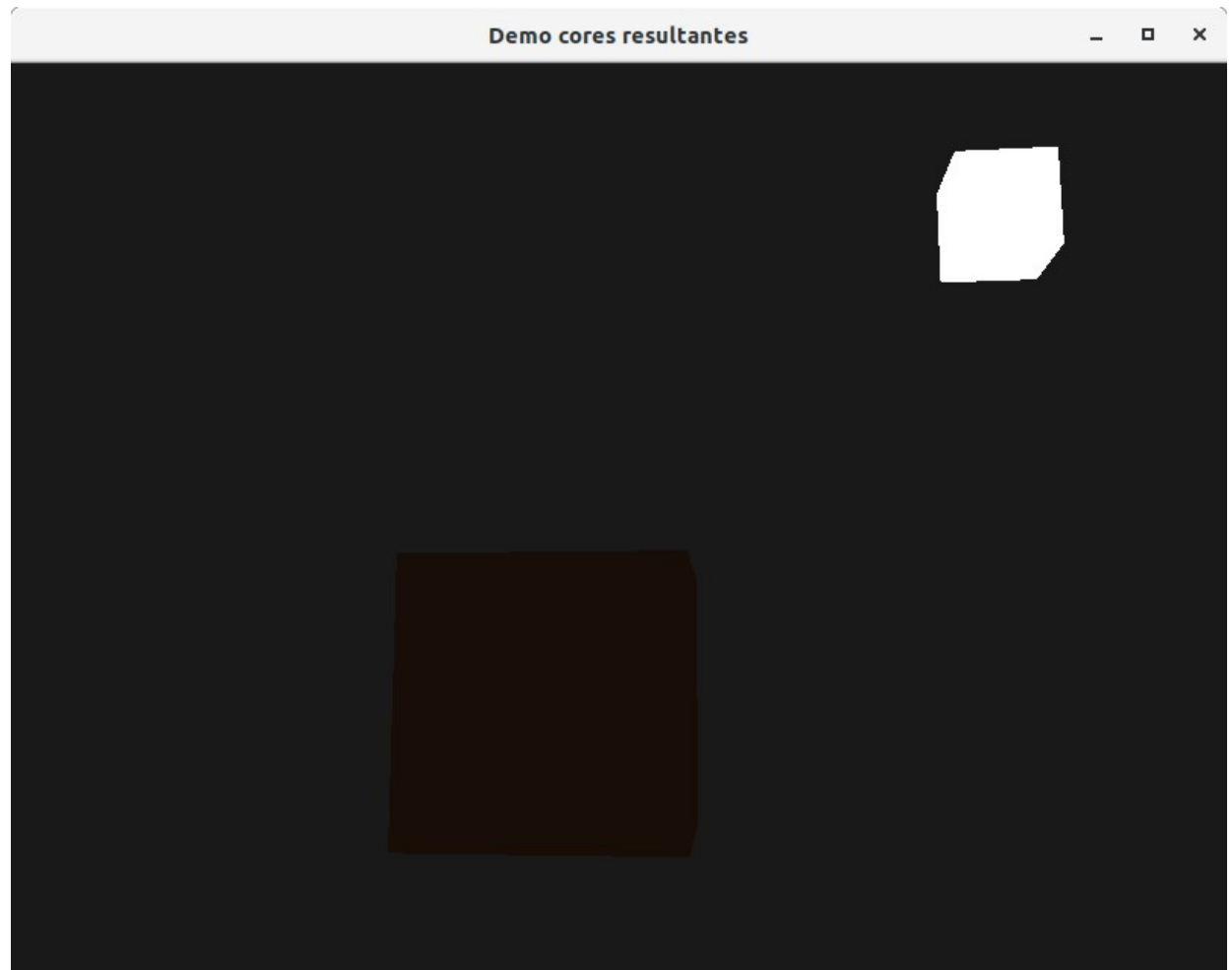
Iluminação Ambiente

- No *fragment shader*:

```
void main()  
{  
    float ambientStrength = 0.1;  
    vec3 ambient = ambientStrength * lightColor;  
  
    vec3 result = ambient * objectColor;  
    FragColor = vec4(result, 1.0);  
}
```


Executando...

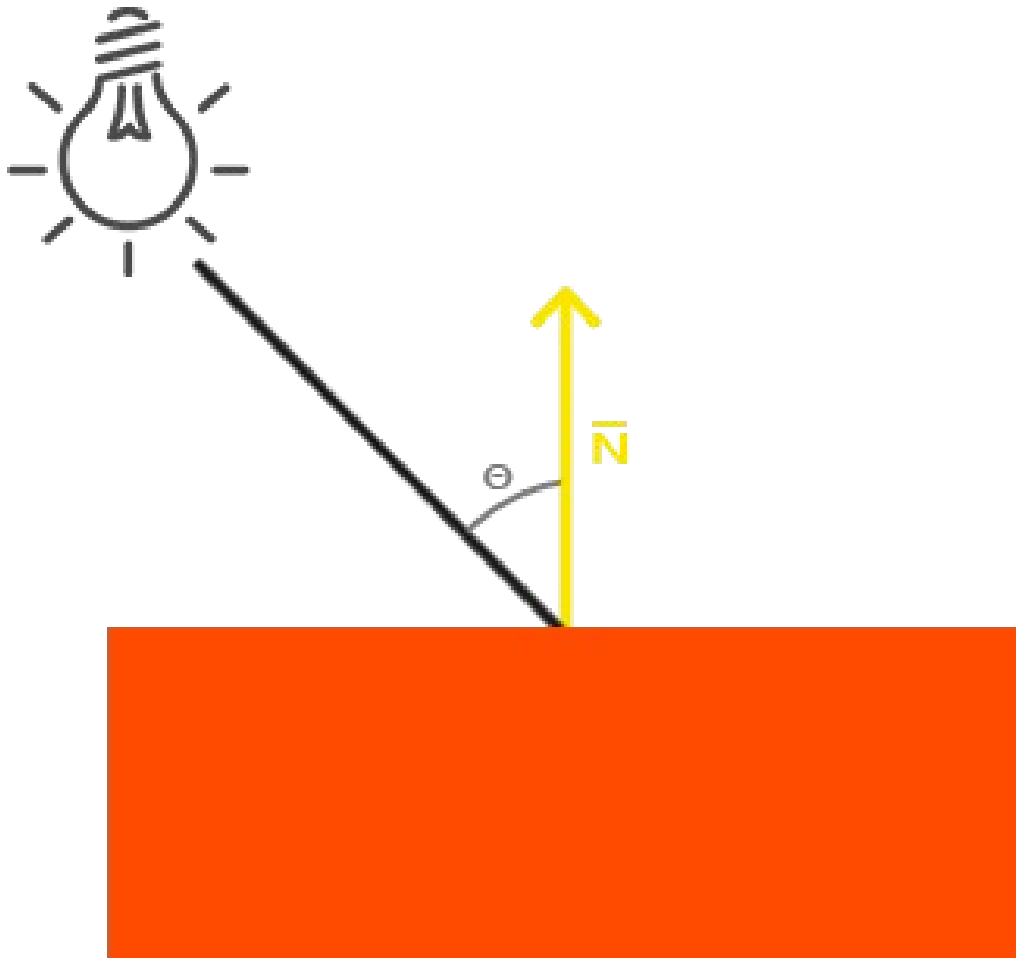
- Note que o primeiro estágio de iluminação foi agora aplicado com sucesso ao objeto.
- O objeto é bastante escuro, já que **somente a iluminação ambiente é aplicada**
- (observe que o cubo de luz não é afetado porque usamos um shader diferente).



Roteiro

- Cores
- Iluminação Básica
 - Iluminação Ambiente
 - Iluminação Difusa

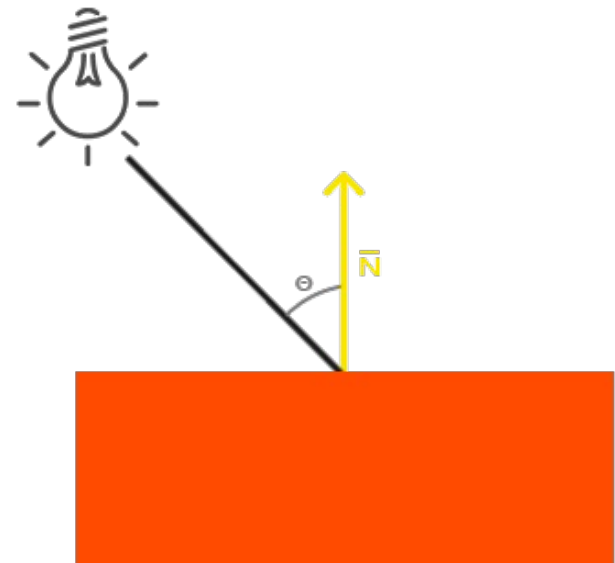
Iluminação Difusa



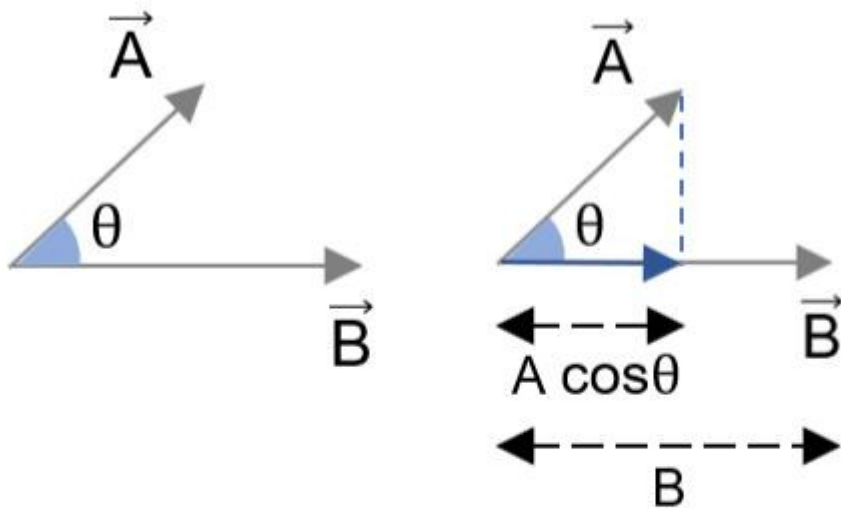
- A iluminação difusa dá ao objeto mais brilho quanto mais perto seus fragmentos estão alinhados aos raios de luz de uma fonte.

Iluminação Difusa

- Observa-se em que ângulo o raio de luz toca o fragmento: Se o raio de luz for perpendicular à superfície do objeto, a luz terá o maior impacto.
- Para medir o ângulo entre o raio de luz e o fragmento, usamos algo chamado de **vetor normal**, que **é um vetor perpendicular à superfície do fragmento** (aqui representado como uma seta amarela);
- O ângulo entre os dois vetores pode ser calculado com o **produto escalar**.



Produto Escalar



Logo

$$\cos \theta = \frac{\vec{A} \cdot \vec{B}}{AB}$$

$$\vec{A} \cdot \vec{B} = AB \cos \theta$$

↳ Ângulo formado entre os vetores $\vec{A}$ e $\vec{B}$

Produto Escalar

Podemos generalizar para vetores com três dimensões

$$\vec{A} = (q, r, s)$$

$$\vec{B} = (t, u, v)$$

O produto escalar $\vec{A} \cdot \vec{B}$ é dado por

$$\vec{A} \cdot \vec{B} = (qt) + (ru) + (sv)$$

Na biblioteca **GLM** existe a função **dot**, que calcula o produto escalar entre dois Vetores:

```
glm::vec3 vec1(1.0f,0.0f,0.0f);  
glm::vec3 vec2(1.0f,0.8f,0.0f);  
double result = glm::dot(glm::normalize(vec1), glm::normalize(vec2));
```

Produto Escalar e Luz

- Quanto menor o ângulo entre dois vetores unitários, mais o produto escalar se aproxima de 1.
- Quando o ângulo entre os dois vetores é de 90 graus, o produto escalar se torna 0.
- O mesmo se aplica para θ : Quanto maior o ângulo θ , menos impacto a luz deve ter sobre a cor do fragmento.

Produto Escalar e Luz

- O produto escalar resultante, portanto, retorna um valor escalar que podemos usar para calcular o impacto da luz na cor do fragmento, resultando em fragmentos iluminados de forma diferente com base em sua orientação em relação à luz.

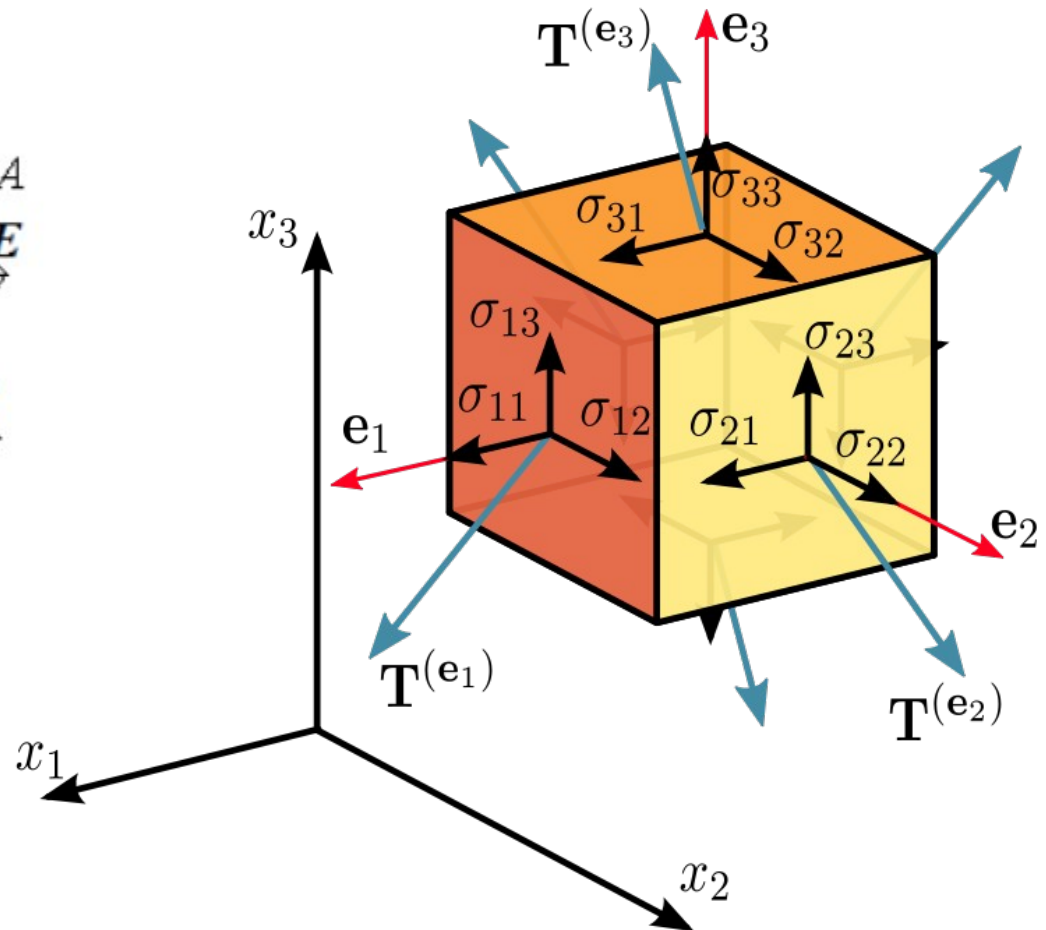
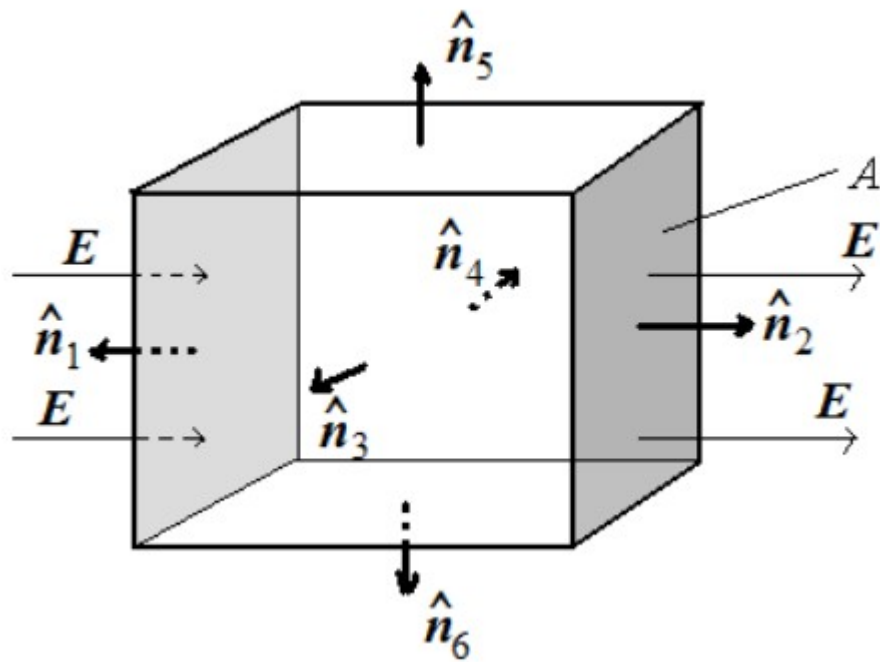
Cálculo da Iluminação Difusa

- Para calcular a iluminação difusa são necessários:
 - **Vetor normal**: um vetor que é perpendicular à superfície do vértice.
 - **O raio de luz direcionado**: um vetor de direção que é o vetor de diferença entre a posição da luz e a posição do fragmento.
 - Para calcular este raio de luz, precisamos do **vetor de posição da luz** e do **vetor de posição do fragmento**.

Vetor Normal

- **Um vetor normal é um vetor (unitário) perpendicular à superfície de um vértice.**
- Como um vértice por si só não tem superfície (é apenas um único ponto no espaço), recuperamos um vetor normal usando seus vértices circundantes para descobrir a superfície do vértice.
- Podemos calcular os vetores normais para todos os vértices do cubo usando o produto vetorial, mas como um cubo 3D não é uma forma complicada, podemos simplesmente adicioná-los manualmente aos dados do vértice.

Vetor Normal



Vetor Normal

- Um exemplo da matriz de dados de vértice contendo os vetores normais é demonstrado aqui.

float vertices[] = {

-0.5f, -0.5f, -0.5f,	0.0f, 0.0f, -1.0f,
0.5f, -0.5f, -0.5f,	0.0f, 0.0f, -1.0f,
0.5f, 0.5f, -0.5f,	0.0f, 0.0f, -1.0f,
0.5f, 0.5f, -0.5f,	0.0f, 0.0f, -1.0f,
-0.5f, 0.5f, -0.5f,	0.0f, 0.0f, -1.0f,
-0.5f, -0.5f, -0.5f,	0.0f, 0.0f, -1.0f,
-0.5f, -0.5f, 0.5f,	0.0f, 0.0f, 1.0f,
0.5f, -0.5f, 0.5f,	0.0f, 0.0f, 1.0f,
0.5f, 0.5f, 0.5f,	0.0f, 0.0f, 1.0f,
0.5f, 0.5f, 0.5f,	0.0f, 0.0f, 1.0f,
-0.5f, 0.5f, 0.5f,	0.0f, 0.0f, 1.0f,
-0.5f, -0.5f, 0.5f,	0.0f, 0.0f, 1.0f,
-0.5f, 0.5f, 0.5f,	-1.0f, 0.0f, 0.0f,
-0.5f, 0.5f, -0.5f,	-1.0f, 0.0f, 0.0f,
-0.5f, -0.5f, -0.5f,	-1.0f, 0.0f, 0.0f,
-0.5f, -0.5f, -0.5f,	-1.0f, 0.0f, 0.0f,
-0.5f, -0.5f, 0.5f,	-1.0f, 0.0f, 0.0f,
-0.5f, 0.5f, 0.5f,	-1.0f, 0.0f, 0.0f,

Computação Gráfica

Vértices

Vetores Normais

Vetor normal : Shaders

- Já que adicionamos dados extras ao vetor de vértices precisamos ajustar o shader de vértices do cubo:

```
#version 330 core
```

```
layout (location = 0) in vec3 aPos;
```

```
layout (location = 1) in vec3 aNormal;
```

```
...
```

Vetor normal : Shaders

- E os ponteiros de atributo dos vértices também devem ser ajustados:

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);  
glEnableVertexAttribArray(0);
```

Vetor normal : Shaders

- Todos os cálculos de iluminação são feitos no shader de fragmento, portanto, precisamos encaminhar os vetores normais do shader de vértice para o shader de fragmento.

```
out vec3 Normal;
```

```
void main()  
{  
    gl_Position = projection * view * model * vec4(aPos, 1.0);  
    Normal = aNormal;  
}
```

Vetor normal : Shaders

- No shader de fragmentos, declara-se uma variável para receber o vetor normal, e uma variável ***uniform*** para receber o vetor de direção da luz

`in vec3 Normal;`

`uniform vec3 lightPos;`

- O vetor de direção da luz pode ser enviado dentro do loop principal (se a luz se movimentar), ou fora, caso ela fique estática

OpenGL : Calculando a luz difusa

- Agora que se tem o vetor normal para cada vértice, é necessário informar o vetor de posição da luz e do vetor de posição do fragmento.
- Como a posição da luz é uma única variável estática, podemos enviá-la por ***uniform*** para o shader de fragmentos

```
lightingShader.setVec3("lightPos", lightPos);
```

OpenGL : Calculando a luz difusa

- A última coisa de que precisamos é a posição real do fragmento.
- Faremos todos os cálculos de iluminação no **espaço mundial**, portanto, queremos uma posição de vértice que esteja no espaço mundial primeiro.
- Podemos fazer isso multiplicando o atributo de posição do vértice apenas pela matriz do modelo (não pela matriz de visualização e projeção) para transformá-la em coordenadas do espaço mundial.
- Isso pode ser facilmente realizado no vertex shader, então vamos declarar uma variável de saída e calcular suas coordenadas de espaço mundial:

```
out vec3 FragPos;  
out vec3 Normal;
```

```
void main()  
{  
    gl_Position = projection * view * model * vec4(aPos, 1.0);  
    FragPos = vec3(model * vec4(aPos, 1.0));  
    Normal = aNormal;  
}
```

OpenGL : Calculando a luz difusa

- Por fim se adiciona a variável de entrada FragPos ao shader de fragmentos

`in vec3 FragPos;`

- Esta variável será interpolada a partir dos 3 vetores de posição mundial do triângulo para formar o vetor FragPos que é a posição mundial por fragmento.
- Agora que todas as variáveis necessárias estão definidas, podemos iniciar os cálculos de iluminação.

OpenGL : Calculando a luz difusa

- Calcula-se o vetor de direção entre a fonte de luz e a posição do fragmento.
- O vetor de direção da luz é o vetor de diferença entre o vetor de posição da luz e o vetor de posição do fragmento.
- Pode-se calcular essa diferença subtraindo os dois vetores um do outro.
- É necessário ter certeza de que todos os vetores relevantes terminam como **vetores unitários**, então normaliza-se o vetor de direção normal e resultante:

```
vec3 norm = normalize(Normal);
```

```
vec3 lightDir = normalize(lightPos - FragPos);
```

OpenGL : Calculando a luz difusa

- Em seguida, **calcula-se o impacto difuso da luz no fragmento atual**, tomando o produto escalar entre os vetores `norm` e `lightDir`.
- O **valor resultante é então multiplicado pela cor da luz** para obter o componente difuso, resultando em um componente difuso mais escuro quanto maior o ângulo entre os dois vetores:

```
float diff = max(dot(norm, lightDir), 0.0);  
vec3 diffuse = diff * lightColor;
```

OpenGL : Calculando a luz difusa

- Se o ângulo entre os dois vetores for maior que 90 graus, o resultado do produto escalar se tornará negativo e terminaremos com um componente difuso negativo.
- Por essa razão, usa-se a função `max` que retorna o maior de seus dois parâmetros para garantir que o componente difuso (e, portanto, as cores) nunca se torne negativo.
- A iluminação para cores negativas não está realmente definida

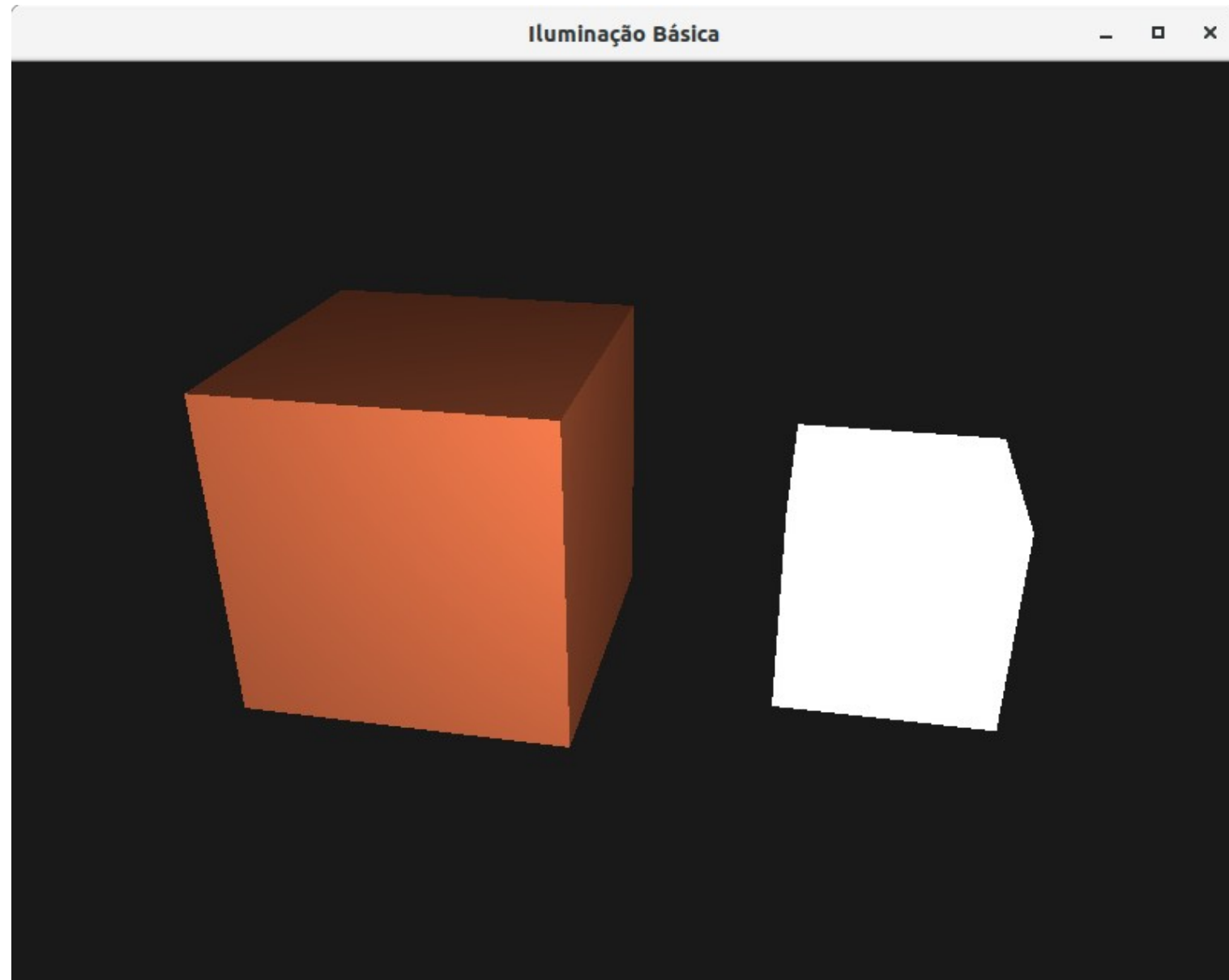
OpenGL : Calculando a luz difusa

- Agora que temos um componente **ambiente** e um **difuso**, **adiciona-se as duas cores uma à outra** e, em seguida, **multiplica-se o resultado pela cor do objeto** para obter a cor de saída do fragmento resultante:

```
vec3 result = (ambient + diffuse) * objectColor;  
FragColor = vec4(result, 1.0);
```

Resultado

- O resultado que devemos obter é o seguinte:



Exemplo

- O código que gera o resultado anterior está disponível no ambiente virtual em um projeto chamado **GLFW_Iluminacao_1**

Dúvidas?



Atividade 1

- Faça o download e execute em seu computador o projeto **GLFW_Iluminacao_0**.
- Depois, troque a cor do cubo maior para outra de sua preferência e observe os efeitos na cor do cubo quando você pressiona as teclas **R**, **F**, **T**, **G**, **Y** e **H** para mudar a cor da luz emitida.

Atividade 2

- Faça o download e execute em seu computador o projeto **GLFW_Iluminacao_1**.
- Implemente nesse projeto uma lógica que **faça a luz girar** em círculos ao redor do cubo, e verifique o comportamento das luzes.